Rapport de projet : Stack d'observabilité
Partie conception

1. Contexte

Dans une architecture microservices, une requête traverse plusieurs services (Client → API Gateway → Authentification → Réservation → Paiement → Notification), contrairement à un monolithe où tout se passe dans un seul processus. Il devient alors difficile de savoir quel service a planté, où se situe le goulot d'étranglement ou pourquoi une requête est lente.

L'objectif du projet est de construire une petite application microservices en Java (Spring Boot) et d'y mettre en place une stack d'observabilité : les logs, les métriques et les traces sont collectés avec OpenTelemetry, puis restitués dans Grafana. Ce projet nous permet de comprendre le fonctionnement des microservices et de découvrir des outils utilisés en entreprise.

2. Périmètre du projet

Ce qu'on couvre :

Pour pouvoir utiliser notre stack d'observabilité et s'assurer qu'elle soit efficace, on doit avoir

- **Service de point d'entrée (api gateway) :** Le client ne connaît que l'adresse du gateway, les autres services sont cachés et le point d'entrée permet de diriger le client au bon endroit, c'est le routage

- **Service d'authentification :** Le service gère la connexion des utilisateurs, il vérifie qu'un utilisateur a bien un compte (que ses informations de connexion sont bien enregistrées dans la base de données), sinon l'utilisateur doit s'inscrire et le système enregistre ses informations dans la base de données.

- **Service de réservation** : Une fois authentifié, un utilisateur peut réserver une entité (un livre par exemple si on est dans une application de librairie), l'entité sera enregistrée dans le panier de l'utilisateur, il pourra ensuite annuler sa réservation et la confirmer en procédant au paiement.

-

- Service de paiement fictif : On fait une simulation d'un système de paiement: le système va demander les informations de paiement (numéro de carte, date d'expiration et cryptogramme visuel) et vérifie si la carte est valide (pas expirer, bien 16 chiffres sur le numéro de carte, et 3 pour le cryptogramme). Ensuite le système valide le paiement en créant une commande dans la base données avec ce qu'il y avait dans le panier et vide le panier.

- Service de notification : Une fois le paiement accepté, le service de notification prévient l'utilisateur si c'est confirmé ou non, dans le projet on simule ce message via un log, on peut le faire via appel synchrone REST (réservation → http → notification) ou ASYNCHRONE avec une file de messages comme RabbitMQ (à confirmer avec le prof le choix à faire)

Ce que l'on couvre en complément :

- **Sécurité de la stack** : On peut sécuriser les services avec un jeton JWT, l'utilisateur s'identifie une fois et reçoit un jeton qui se présentera à chaque requête, c'est d'ailleurs le service d'api gateway qui vérifie le jeton, s'il est invalide le service rejettera la requête.

Hors périmètre :

- Un vrai système de paiement et un vrai envoi d'e-mails
- Une interface graphique pour le client
- Pas de mise en production via Kubernetes

3. Existant

Le projet partant de zéro, il est nécessaire de mettre en place les outils sur lesquels on va s'appuyer et pourquoi.

Les outils retenus :

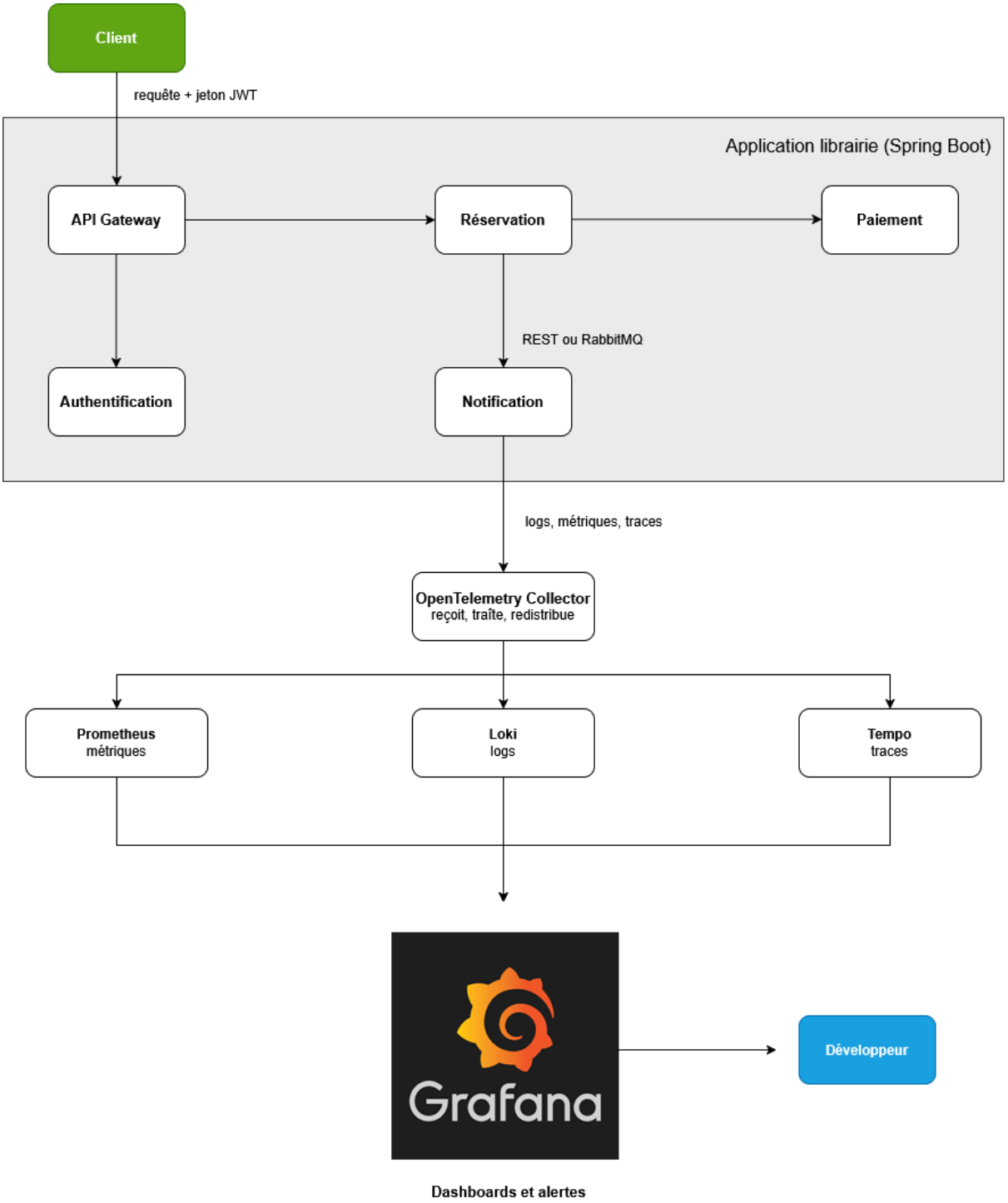
Pour l'application :

- **Java et Spring Boot** : Spring Boot est un framework Java qui facilite la création de services web autonomes (serveur intégré, configuration automatique. Chaque microservice est une application Spring Boot
- **Spring Cloud Gateway** : Bibliothèque de l'écosystème Spring qui sert à construire l'API Gateway (routage et filtres, dont la vérification du jeton).
- **PostgreSQL** : Base de données relationnelle utilisée par les services qui stockent des données.
- **RabbitMQ** (Si la notification asynchrone est retenue après la réunion) : Le service de réservation dépose un message, le service de notification le traite quand il est disponible
- **Docker et Docker Compose** : Docker place chaque service dans un conteneur isolé ; Docker Compose décrit tous les conteneurs dans un seul fichier et les lance avec une seule commande

Pour l'observabilité :

- **OpenTelemetry / OpenTelemetry Collector**: OpenTelemetry est un outil qui permet de générer, exporter et collecter des données de télémétrie. Il va nous servir dans notre projet après chaque requête à générer les données de télémétrie, ensuite les envoyées à OpenTelemetry Collector,un composant d'OpenTelemetry, qui lui va faire le tri et les envoyées au bon endroit (Loki, Tempo, Prometheus).
- **Loki**: va permettre de stocker les logs provenant d'OpenTelemetry Collector et les envoyer à grafana lorsqu'il les demande
- **Tempo**: va permettre de stocker les traces et les envoyer à grafana lorsqu'il en a besoin
- **Prometheus**: Prometheus est une base de données temporelle qui collecte les métriques régulièrement, les analyse grâce à son propre langage appelé **PromQL** et génère des alertes lorsqu'il détecte une anomalie via son module **AlertManager**
- **Grafana** : Grafana va interroger Loki, Tempo et Prometheus et afficher les données sous forme de graphiques dans des dashboards afin de pouvoir les

4. Architecture Global



5. Acteurs

Un acteur est un rôle extérieur au système qui interagit avec lui :

Acteur	Type	Rôle
Développeur, DevOps	Principal, humain	Instrumenter les outils, utiliser Grafana pour surveiller l'application, diagnostiquer les erreurs et les lenteurs etc, il reçoit aussi les alertes
Client de démonstration	Secondaire, humain	S'inscrit, se connecte, réserve et paie (via l'API REST), il ne voit pas la stack d'observabilité mais c'est son activité qui produit les données observées
Système d'alerte (Grafana)	Secondaire, système	Surveille les métriques en continu et prévient le développeur lorsqu'un seuil est dépassé

6. Cas d'usage

Cas d'usage	Acteur principal	Résultat attendu
Consulter l'état global du système	Exploitation / DevOps	Un tableau de bord présente le trafic, les erreurs et les temps de réponse.
Suivre une requête entre	Développeur	Une trace montre les services traversés, les

les services	Développeur	opérations effectuées et leur durée.
Rechercher les logs d’une requête	Développeur	Les logs associés à un trace ID peuvent être retrouvés.
Identifier un ralentissement		